space—commonly referred to as **containers** or **jails**. There are many differences with hypervisor-based virtualization. The main one is that while a container may *look* like an isolated computer (with its own devices, its own memory, etc.), the underlying operating system is fixed and cannot be replaced by another. A second important difference is that since all containers use the same underlying operating system (and hence share its resources), the isolation between containers is less complete than that between virtual machines. A third difference is that, precisely because the containers directly use the services of a single operating system, they are often more lightweight and efficient than hypervisor-based solutions. In this chapter, our main focus is on hypervisor-based virtualization, but we will discuss OS-level virtualization also.

7.1 HISTORY

With all the hype surrounding virtualization in recent years, we sometimes forget that by Internet standards virtual machines are ancient. As early as the 1960s. IBM experimented with not just one but two independently developed hypervisors: **SIMMON** and **CP-40**. While CP-40 was a research project, it was reimplemented as **CP-67** to form the control program of **CP/CMS**, a virtual machine operating system for the IBM System/360 Model 67. Later, it was reimplemented again and released as **VM/370** for the System/370 series in 1972. The System/370 line was replaced by IBM in the 1990s by the System/390. This was essentially just a name change since the underlying architecture remained the same for reasons of backward compatibility. Of course, the hardware technology was greatly improved and the newer machines were bigger and faster than the older ones, but as far as virtualization was concerned, nothing changed. In 2000, IBM released the z-series, which supported 64-bit virtual address spaces but was otherwise backward compatible with the System/360. All of these systems supported virtualization decades before it became popular on the x86.

In 1974, two computer scientists at UCLA, Gerald Popek and Robert Goldberg, published a seminal paper ("Formal Requirements for Virtualizable Third Generation Architectures") that listed exactly what conditions a computer architecture should satisfy in order to support virtualization efficiently (Popek and Goldberg, 1974). It is impossible to write a chapter on virtualization without referring to their work and terminology. Famously, the well-known x86 architecture that also originated in the 1970s did not meet these requirements for decades. It was not the only one. Nearly every architecture since the mainframe also failed the test. The 1970s were very productive, seeing also the birth of UNIX, Ethernet, the Cray-1, Microsoft, and Apple—so, despite what your parents may say, the 1970s were not just about disco!

In fact, the real **Disco** revolution started in the 1990s, when researchers at Stanford University developed a new hypervisor by that name and went on to found **VMware**, a virtualization giant that offers type 1 and type 2 hypervisors and now

rakes in billions of dollars in revenue (Bugnion et al., 1997, Bugnion et al., 2012). Incidentally, the distinction between "type 1" and "type 2" hypervisors is also from the seventies (Goldberg, 1972). VMware introduced its first virtualization solution for x86 in 1999. In its wake other products followed: **Xen**, **KVM**, **VirtualBox, Hyper-V, Parallels**, and many others. It seems the time was right for virtualization, even though the theory had been nailed down in 1974 and for decades IBM had been selling computers that supported—and heavily used—virtualization. In 1999, it became popular among the masses, but new it was not, despite the massive attention it suddenly gained.

OS-level virtualization, while not as old as hypervisors, has a history that stretches back quite some time also. In 1979, UNIX v7 introduced a new system call: chroot (change root). The system call takes as its single argument a path name, for instance /home/hjb/my_new_root, which is where it will create a new "root" directory for the current process and all of its children. Thus, when the process reads a file /README.txt (a file in the root directory), it really accesses /home/hjb/my_new_root/README.txt. In other words, the operating system has created a separate environment on disk to which the process is confined. It cannot access files from directories other than those in the subtree below the new root (and we had better make sure that all files the process ever needs will be in this subtree, because they are literally all it can access).

A kind soul might consider this enough to be called a virtual environment, but clearly it is an exceedingly poor man's version of that. For instance, while the visibility of the file system is limited to a subtree, there is no isolation of processes or their privileges. Thus, a process inside the subtree can still send signals to processes outside the subtree. Similarly, a process with administrator (or "root") privileges cannot be properly locked inside the chroot subtree: having root privileges, it can do *anything*, including breaking out of the confined file name space. Another problem is that processes in different chroot subtrees still have access to all the IP addresses assigned to the machine and there is no isolation between packets sent from this process and those sent by processes in other subtrees. In other words, these are not virtual machines at all.

In 2000, Poul-Henning Kamp and Robert Watson extended the chroot isolation in the FreeBSD operating system to create what they referred to as **FreeBSD Jails** (Kamp and Watson, 2000). They partitioned resources much more pervasively: Jails had their own file system name spaces, their own IP addresses, their own (limited) root processes, etc. Their elegant solution was hugely influential and soon similar features appeared in other operating systems, often partitioning even more resources, such as memory or CPU usage. In 2008, Linux Containers (LXC) was released. Based on an earlier resource partitioning project at Google, LXC offers partitioning of resources of a collection of processes through containers. However, the popularity of containers really exploded with the launch of **Docker** in 2013. Docker uses OS-level virtualization to allow software to be packaged in containers that hold all the code, libraries, and configuration files needed to run it.

Many people use the term "virtualization" to refer exclusively to hypervisor-based solutions, and use the term "containerization" to talk about OS-level virtualization. While we emphasize that in reality both are forms of virtualization, we reluctantly adopt the same convention in this chapter. So, unless we explicitly say otherwise (e.g., when we talk about containers), you may assume that henceforth virtualization refers to the hypervisor variety.

7.2 REQUIREMENTS FOR VIRTUALIZATION

If we leave OS-level virtualization aside, it is important that virtual machines act just like the real McCoy. In particular, it must be possible to boot them like real machines and install arbitrary operating systems on them, just as can be done on the real hardware. It is the task of the hypervisor to provide this illusion and to do it efficiently. Indeed, hypervisors should score well in three dimensions:

1. **Safety**: The hypervisor should have full control of the virtualized resources.
2. **Fidelity**: The behavior of a program on a virtual machine should be identical to that of the same program running on bare hardware.
3. **Efficiency**: Much of the code in the virtual machine should run without intervention by the hypervisor.

An unquestionably safe way to execute the instructions is to consider each instruction in turn in an **interpreter** (such as Bochs) and perform exactly what is needed for that instruction. Some instructions can be executed directly, but not too many. For instance, the interpreter may be able to execute an INC (increment) instruction simply as is, but instructions that are not safe to execute directly must be simulated by the interpreter. For instance, we cannot really allow the guest operating system to disable interrupts for the entire machine or modify the page-table mappings. The trick is to make the operating system on top of the hypervisor think that it has disabled interrupts, or changed the machine's page mappings. We will see how this is done later. For now, we just want to say that the interpreter may be safe, and if carefully implemented, perhaps even hi-fi, but the performance sucks. To also satisfy the performance criterion, we will see that VMMs try to execute most of the code directly.

Now let us turn to fidelity. Virtualization has long been a problem on the x86 architecture due to defects in the Intel 386 architecture that were automatically carried forward into new CPUs for 20 years in the name of backward compatibility. In a nutshell, every CPU with kernel mode and user mode has a set of instructions that behave differently when executed in kernel mode than when executed in user mode. These include instructions that do I/O, change the MMU settings, and so on. Popek and Goldberg called these **sensitive instructions**. There is also a set of